

Experiment 3: polyalphabetic substitution cipher

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

char matrix[5][5];

void createMatrix(char key[]) {
    int used[26] = {0};
    int i, j, k = 0;
    char ch;

    used['J' - 'A'] = 1;

    for (i = 0; key[i] != '\0'; i++) {
        ch = toupper(key[i]);

        if (ch == 'J')
            ch = 'I';

        if (ch >= 'A' && ch <= 'Z' && !used[ch - 'A']) {
            matrix[k / 5][k % 5] = ch;
            used[ch - 'A'] = 1;
            k++;
        }
    }

    for (ch = 'A'; ch <= 'Z'; ch++) {
        if (ch == 'J')
            continue;
```

```

        if (!used[ch - 'A']) {
            matrix[k / 5][k % 5] = ch;
            used[ch - 'A'] = 1;
            k++;
        }
    }
}

```

```

void findPosition(char ch, int *row, int *col) {
    int i, j;

    if (ch == 'J')
        ch = 'I';

    for (i = 0; i < 5; i++) {
        for (j = 0; j < 5; j++) {
            if (matrix[i][j] == ch) {
                *row = i;
                *col = j;
                return;
            }
        }
    }
}

```

```

void encryptPair(char a, char b, char *x, char *y) {
    int r1, c1, r2, c2;

    findPosition(a, &r1, &c1);
    findPosition(b, &r2, &c2);
}

```

```
if (r1 == r2) {  
    *x = matrix[r1][(c1 + 1) % 5];  
    *y = matrix[r2][(c2 + 1) % 5];  
}
```

```
else if (c1 == c2) {  
    *x = matrix[(r1 + 1) % 5][c1];  
    *y = matrix[(r2 + 1) % 5][c2];  
}
```

```
else {  
    *x = matrix[r1][c2];  
    *y = matrix[r2][c1];  
}  
}
```

```
int main() {  
    char key[100], plaintext[1000], clean[1000];  
    char a, b, x, y;  
    int i, j = 0;  
  
    printf("Enter the keyword: ");  
    scanf("%s", key);  
  
    printf("Enter the plaintext: ");  
    scanf(" %[^\n]", plaintext);  
  
    createMatrix(key);
```

```
printf("\nPlayfair Matrix:\n");
for (i = 0; i < 5; i++) {
    for (j = 0; j < 5; j++)
        printf("%c ", matrix[i][j]);
    printf("\n");
}
```

```
for (i = 0; plaintext[i] != '\0'; i++) {
    if (isalpha(plaintext[i])) {
        clean[j++] = toupper(plaintext[i]);
    }
}
clean[j] = '\0';
```

```
for (i = 0; clean[i] != '\0'; i++) {
    if (clean[i] == 'J')
        clean[i] = 'I';
}
```

```
j = 0;
for (i = 0; clean[i] != '\0'; i++) {
    if (j > 0 && clean[i] == clean[j - 1]) {
        clean[j++] = 'X';
    }
    clean[j++] = clean[i];
}
clean[j] = '\0';
```

```

if (j % 2 != 0) {
    clean[j++] = 'X';
    clean[j] = '\0';
}

printf("\nEncrypted text: ");

for (i = 0; i < j; i += 2) {
    a = clean[i];
    b = clean[i + 1];

    encryptPair(a, b, &x, &y);

    printf("%c%c", x, y);
}

printf("\n");

return 0;
}

```

```

C:\Users\kailash\Downloads\...
Enter the keyword: MONARCHY
Enter the plaintext: INSTRUMENTS

Playfair Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Encrypted text:
-----
Process exited after 17.7 seconds with return value 0
Press any key to continue . . . |

```